

Design document

Context & current state -

The system is an early-stage build that has outgrown its original assumptions. Everything runs on EC2 with manual SSH deployments and manual infrastructure changes. There is no IaC, no CI/CD, and no real access management. Monitoring is minimal, with no alerting to the team when something goes wrong, so issues are often noticed only after users are affected.

Two specific failure modes are causing the current outages and slowdowns, both of which are triggered when large clients run expensive tasks. First, these tasks run in the same process that serves user requests, leading to high memory usage and swap thrashing that degrade the entire application. Second, the same tasks write hundreds of thousands of rows to the database in tight loops, overwhelming the primary and slowing or stalling production traffic. The load is spiky rather than steady, so the system has no way to absorb a burst without affecting everyone.

The priority is therefore visibility, establishing observability to confirm where the bottlenecks actually are before making structural changes.

Guiding Principles -

I'd stage the work across three horizons rather than rushing a migration. The priority is to understand the system and stop the active bleeding with small, reversible fixes, never breaking changes. In parallel, I'd establish Terraform as the foundation, so that from month one onward every structural change is codified and repeatable rather than manual. Each phase is a deliberate step toward a stable, scalable end state, the north star, with no risky big-bang cutovers along the way.

Requirements & Approach -

Requirement	Approach
Ready for increased demand	IaC (Terraform) as the scalable foundation, plus an autoscaling ECS architecture that grows with load, not manual provisioning
Observable + business analytics	Centralized logs, metrics, and tracing (OTel) across app and infra-level for health, app-level for troubleshooting and analytics
CI/CD	GitHub Actions (no CI server to operate) for build, test, and infra PR checks; expand to signing and prod gating as it matures
Spikes, no downtime	Decouple heavy tasks to an async SQS + worker tier so spikes miss the request path; autoscale web on requests, workers on queue depth
Fixed outbound IP	Private subnets egress through a NAT Gateway + allocated EIP, one predictable, whitelisted source IP
Database scalability	RDS Proxy for connections → read replicas for reads → async queue + batched writes → Aurora (Serverless v2) long-term; sharding last resort
Security	Secrets Manager with rotation + docs; least-privilege IAM (admin for ops, scoped read-only for others), all codified in Terraform

Target architecture -

The target system separates the user-facing request path from heavy background work. Static frontend assets are served from S3 + CloudFront at the edge. Dynamic requests enter through an Application Load Balancer to an ECS Fargate web service running in private subnets. Light, synchronous reads and writes go through RDS Proxy to the database. Expensive tasks are not run in-line; the web service publishes them to an SQS queue, and a separate ECS Fargate worker service consumes them, batching writes through the same RDS Proxy. All outbound traffic to external partner systems leaves through a NAT Gateway with an allocated Elastic IP, giving partners a single fixed source IP to whitelist. The accompanying Terraform repo implements a representative slice of the networking foundation and an auto-scaling ECS service to demonstrate the IaC approach the rest of the plan builds on.

Key design decisions and trade-offs -

ECS Fargate over EKS - Fargate gives container orchestration and autoscaling with zero cluster ops and native AWS integration, which suits a small team with no dedicated platform engineers. EKS is overkill at this stage and adds a control-plane and node-management burden a solo hire shouldn't carry.

Trade-off - less flexibility and a smaller ecosystem than Kubernetes, revisited once the product and team grow enough to justify it.

Fargate over the EC2 launch type - no EC2 hosts to patch, scale, or manage; provision tasks, not servers.

Trade-off - higher per-unit compute cost than raw EC2, accepted in exchange for eliminated operational overhead, and offset by running the stateless workers on Fargate Spot.

SQS + async worker to decouple heavy workloads - moves expensive tasks off the synchronous request path, directly fixing the memory/swap thrashing and the database write storm in the current system; web and workers then scale independently on their own signals.

Trade-off - added moving parts and eventual consistency on job completion; the user receives an "accepted," not an immediate "done."

Static frontend on S3 + CloudFront, not ECS - a React build is static files with no server to run; S3 + CloudFront serves them at the edge, scales infinitely, needs no autoscaling, and costs a fraction of running containers.

Trade-off - essentially none for a static SPA; server-side rendering would change the calculus.

GitHub Actions for CI/CD - the code already lives in GitHub, so integration is native, and there's no CI server to operate, patch, or secure, the same minimize-what-you-run principle behind the Fargate choice.

Trade-off - tighter coupling to the GitHub ecosystem and less raw flexibility than self-hosted Jenkins, which only earns its operational cost with a platform team or on-prem/regulated constraints.

Single deployable service (modular monolith) over microservices - microservices add network calls, API versioning, and distributed-system complexity a small team shouldn't take on prematurely; one service keeps operations simple.

Trade-off - less independent scaling per component is acceptable now, revisited as the team and domains grow. (Code stays in a single repo initially, with a frontend/backend/infra split as a later, separate step.)

Optimize RDS first, defer Aurora - RDS with Proxy, read replicas, and batched writes covers the near-term need without a database migration. Aurora (Serverless v2) is the longer-term target for spiky load, but deferred until compute is stable on ECS, since a database migration is the riskiest cutover and isn't worth rushing.

Trade-off - RDS replicas have higher lag and less seamless failover than Aurora is acceptable until the migration is justified.

CloudWatch first, external observability later - native to AWS, nothing extra to run, covers logs, metrics, and alarms for the current scope. Datadog or similar comes when cost permits and cross-service correlation justifies it.

Trade-off - CloudWatch is clunkier for cross-service dashboards and querying than a dedicated platform.

Basic alerting first, PagerDuty later - CloudWatch alarms to SNS/email ensure the team finds out before users do, cheaply; proper on-call scheduling and escalation are

added as load and budget grow.

Trade-off - no escalation or on-call rotation until then.

Secrets and access management staged - secrets in Secrets Manager and least-privilege IAM from the start, codified in Terraform (admin for ops, scoped read-only for others); formal access-review and rotation processes mature as the team grows.

Trade-off - lighter governance early acceptable at small headcount, tightened deliberately as it grows.

Single NAT Gateway + EIP across AZs - one predictable, whitelisted egress IP for partner systems, with a single NAT chosen for cost.

Trade-off - a single NAT is a single point of failure, and an AZ-throughput chokepoint production would run one per AZ. VPC endpoints can cut NAT cost and traffic for AWS-bound calls, but partner egress still routes through the NAT + EIP, since external whitelisting requires a fixed public IP.

Database scalability -

The database problems stem from the same heavy client tasks that overload the app tier: they drive a large volume of write operations against RDS, while also causing high memory usage in the application itself. This section addresses the database side; the application memory pressure is relieved by the same decoupling described below.

A first, underlying issue is how the application connects to RDS. EC2 instances connect directly to the database, which isn't good practice because connections aren't managed consistently. Connections can be left open and never closed properly, and over time, the app keeps opening new ones, each adding load on RDS. RDS also doesn't scale compute on its own; the instance class is fixed until changed manually, so this has to be planned rather than assumed.

The lowest-hanging fix is to route connections through RDS Proxy. This won't solve the whole database problem, but it pools and bounds connections so an autoscaling fleet can't exhaust RDS and autoscaling alone wouldn't fix that; it would make it worse. Before that, the heavy writes themselves should be batched (bulk insert) rather than written row by row, since fixing how we write is the cheapest win.

Next, a read replica takes read traffic off the primary; important or read-after-write-sensitive operations stay on the primary. The trade-off is replica lag, so only lag-tolerant reads go to the replica.

We also introduce queues and workers to decouple quick database tasks from heavy ones that don't need an immediate response, which is what takes the write-storm load off the primary and, in turn, relieves the application memory pressure. Together, these are the near-term database optimizations.

I'd deliberately avoid a quick database migration. Migrations are challenging and risk data loss and aren't worth rushing while the rest of the platform is still stabilizing. The end goal is a database that handles more on its own, like Aurora (Serverless v2 for spiky load), but given the cost and migration risk, I'd plan that only once we reach a scale that justifies it. If load eventually outgrows a single primary, sharding is an option, but it adds significant complexity, so I wouldn't take that route unless it's genuinely required.

Observability & Operational Readiness -

Logs - There's effectively no logging today. ECS supports CloudWatch logging natively, so the first step is centralizing application logs there to surface failures and errors. As the platform grows, logging can move to a dedicated observability tool like Datadog, which needs a log-shipping mechanism but consolidates logs from AWS and elsewhere in one place. CloudWatch first (nothing extra to run), Datadog when cost and scale justify it.

Metrics - No metrics today either. CloudWatch covers this natively across ECS, the database, the queues, and other resources, giving insight into infra-level health and early warning of failures.

Traces - Tracing is the more involved pillar. It can be handled with an ADOT collector as a sidecar in ECS, the same topology I've run before, shipping logs to Splunk via Fluent Bit and FireLens. The key difference is that traces require instrumenting the application with the OpenTelemetry SDK to actually produce spans; the collector only ships what the app emits. So unlike log shipping, which is passive, tracing needs application-level changes.

SLOs / performance targets -

- API latency: p99 under 200 ms
- Error rate: 5xx under 1% of requests
- Queue health: alarm on oldest-message-age beyond ~30 minutes (target depends on job urgency)
- DB health: connection utilization under 70% of max
- Availability: 99.9% to start, with a path to 99.95%+ as redundancy matures (per-AZ NAT, Aurora multi-AZ)

Alarms - Focus on the major components for a minimal setup: SQS oldest-message-age (>30 min) and queue/DLQ size; ECS task and service failures across metrics; database cluster health and connection metrics; and UI/endpoint reachability. No external service needed at first: CloudWatch alarms to SNS, then email to the team. As the team and platform grow, PagerDuty adds on-call rostering and incident management. Thresholds shouldn't be set arbitrarily; they need to be meaningful and revisited periodically against traffic patterns, or they create on-call noise and fatigue.

Deployment & rollback - I wouldn't make deployment fully automatic at first; there has to be a human in the loop. As the pipeline and environments mature, lower environments can go fully automated and be observed; production stays gated until there's a proven, fast rollback process for outages. Since the platform runs on ECS, blue/green deployment lets a release roll back to the last stable task definition automatically if a health check or other failure occurs. On the async side, if an ECS worker can't process an SQS message, there's a retry mechanism, and after a set number of retries, the message lands in a DLQ held for 14 days initially and manually re-driven once the worker issue is resolved.

Autoscaling - Per the proposed architecture, the frontend (and web tier) scales on request volume; heavy work is flattened by the queue, which drives the ECS worker tier to scale on queue depth so messages don't pile up in SQS.

CI/CD -

Tool choice - I'd choose the option a single person can maintain easily, GitHub Actions over Jenkins or similar, to avoid adding operational overhead for a small team. The code already lives in GitHub, so integration is native, and there's no CI server to run, patch, or secure. This is the same minimize-what-you-run principle behind the Fargate choice.

Pipeline separation - I'd keep image building, PR checks, and deployment as separate pipelines, clearer to reason about and manage, and easier for other engineers to use.

- **Image build:** build the image, scan it, and push it to ECR. If multiple regions exist later, replicate them to those registries.
- **PR checks:** run security checks, tfint, and a **build** → **plan** → **apply** in a throwaway smoke environment, then destroy it, confirming the change stands up cleanly and doesn't break existing infra. Pre-commit hooks catch formatting and syntax issues locally first, shifting errors left and reducing pipeline load.
- **Deployment:** separate and gated with human intervention until the platform stabilizes and genuinely needs continuous deployment, which saves the engineering team time resolving issues. Once the base is covered (monitoring, fast rollback, alerting, artifact-driven releases), lower environments can go fully automatic, with production still prompted manually.

Artifact-driven releases - Rather than deploying from local machines or specific branches, releases are artifact-driven: build the image once, sign it, and push it to ECR; every environment then pulls that same signed image, verifies the signature, and deploys it. This gives consistency across environments. The artifact validated in lower environments is exactly what reaches production, so nothing unverified is ever deployed.

Manual prod gate and rollback - covered in the observability section; blue/green with auto-rollback to the last stable task definition, prod gated manually until fast rollback is proven.

Maintainability - This matters most for CI/CD: some tools are easy to install but become a burden to maintain and hard to migrate away from. I'd favour tools that are simple to run and maintain, and reusable pipeline/workflow definitions over

copy-pasted per-service configs, to keep the operational load low as the number of services grows.

Migration & Phased roadmap

Migration approach - The guiding principle is strangler-fig: build the new stack alongside the old one and shift over gradually, so every step is reversible, and there's no big-bang cutover. One change is migrated at a time; never compute and database together.

Compute migration (EC2 → ECS) via weighted ALB - The existing EC2 app and the new ECS service both register behind the load balancer. Traffic shifts by weight rather than all at once, for example, 90/10, then 70/30, then 50/50, then 100% ECS watching error rate and latency at each step. If anything degrades, traffic shifts back to EC2, which is still running, so rollback is near-instant, and an outage is never on the table. EC2 is decommissioned only after ECS has been stable at full traffic. The frontend needs no cutover; it's already static on S3/CloudFront. Crucially, the new ECS tasks connect to the *same* RDS through the same RDS Proxy, so compute moves without touching data.

Database cutover (only when moving to Aurora) - This is a separate, later migration not done alongside the compute move. Because the app connects through the RDS Proxy endpoint rather than directly to the database, the switch is mostly a matter of repointing the Proxy. The approach: create an Aurora copy that stays in sync with the existing RDS, and once it's caught up, briefly pause writes, switch over to Aurora, and resume. Downtime is limited to that short switch-over window, and the original RDS is kept as a fallback until the new setup is validated.

Phased roadmap -

- **Month 1: Understand & stabilize (reversible stopgaps + foundation)** - Learn the system; add monitoring and alerting to surface the real DB and memory failures; clean up access (AWS account access, IAM); replace manual

SSH deploys with scripted deploys pulling credentials from Secrets Manager; add RDS Proxy for connection pooling. In parallel, stand up the Terraform foundation (state backend, core modules) and codify existing infra from here; structural change is IaC.

- **Months 1–3: Build the platform via Terraform** - Build the new ECS platform and migrate EC2 → ECS gradually via the weighted-ALB approach above; improve logging and monitoring across AWS services; introduce basic CI/CD early, so the migration itself runs through a tested, automated pipeline with rollback.
- **Months 3–12 Scale & harden toward the north star** - Database work (read replica, async queue + batched writes for the heavy, memory-bound tasks); CI/CD maturity (gating, signing, multiple environments); secrets, certificate rotation, and access management with documentation; and, if cost and time permit, the Aurora Serverless v2 cutover, external observability (Datadog), and incident management (PagerDuty).

Risks & mitigations -

The migration itself is the biggest risk. If the cutover to ECS goes wrong, the weighted ALB lets me shift traffic straight back to EC2, and the old stack stays live until ECS has proven itself, so a bad cutover never becomes an outage. The database migration carries similar risk, including potential data loss, which is why I defer Aurora until it's clearly justified and use a sync-then-switch approach with the original RDS kept as a fallback.

A few single points of failure are acceptable early on but should be flagged for remediation as the platform matures. The single NAT Gateway is fine for cost now, but production would run one per AZ. More broadly, any SPOF that's tolerable at the start should be revisited and removed as the system grows.

Several risks are already handled by the design. Connection exhaustion under autoscaling is bounded by RDS Proxy. A spike overwhelming the system is absorbed by the async queue, with workers scaling on queue depth so load doesn't hit the request path. Alert noise and on-call fatigue are managed by setting meaningful thresholds and revisiting them against real traffic patterns rather than guessing.

Keeping everything in Terraform with reusable modules and documented changes capturing the reasoning behind each one means the system stays understandable and maintainable by anyone on the team, and easy to hand over or extend.

Security -

- Secrets in Secrets Manager with rotation, documented, and codified in Terraform.
- Least-privilege IAM admin for ops, scoped read-only for everyone else also defined in Terraform.
- Network segmentation: the app and workers run in private subnets; only the ALB is public, and all egress goes through the NAT.
- Terraform separates core and application infrastructure to reduce blast radius, so a routine app change can't touch networking.
- State files are isolated per region and environment, on an S3 backend with DynamoDB locking, so there are no parallel applies and no polluted state; access to the prod state bucket is monitored for security.
- Access to the deployment pipelines themselves is controlled, so only authorized changes reach the environments.